

Análisis crítico – Compresión en Apache (mod_deflate vs mod_brotli)

Eddie Santiago Delgado Campo

Sebastian Leiton Goyes

Christian David Home Acero

Docente

Oscar Hernán Mondragón Martínez

Materia

Servicios Telemáticos

Universidad Autónoma de Occidente

Facultad de Ingeniería

Programa de Ingeniería Informática

2026

7. Brotli vs gzip

¿Cuánto mejora Brotli el ratio frente a gzip sobre texto? ¿En qué tipos de archivo la diferencia es significativa y en cuáles es marginal?

Con los datos que se obtuvieron queda claro que **Brotli le gana a gzip en todos los tipos de archivo de texto** que se probaron y la diferencia varía bastante según qué tan repetitivo sea el contenido. En archivos como el **HTML, SVG o TXT** la **mejora es enorme** ya que en el **HTML gzip nivel 6** deja el archivo en **738 bytes** mientras **Brotli calidad 5** lo deja en solo **174 bytes**, lo que representa casi un **77% menos de peso** frente a lo que había logrado gzip. Algo parecido pasa con el **SVG** donde **gzip llega a 5031 bytes** y **Brotli baja hasta 2220 bytes** y por último, en el **TXT** la diferencia es increíble, logrando un **ahorro casi del 100%**, en el que **gzip llega a 4719 bytes** y **Brotli alcanza en calidad 11 un tamaño de 132 bytes**.

En cambio, en archivos con contenido más variado como el **JavaScript** la **diferencia** sigue existiendo pero se vuelve más **marginal** ya que **gzip nivel 6** deja el [app.js](#) en **84014 bytes** y **Brotli calidad 5** en **79681 bytes**, una mejora de apenas un 5% adicional. Esto tiene sentido porque el diccionario predefinido de Brotli está pensado sobre todo para patrones típicos de HTML, CSS y estructuras de marcado, así que en código JS con nombres de variables y lógica más impredecible ese diccionario ayuda menos.

De esta manera, se puede **concluir** que la **diferencia es más significativa** entre más “**estructurado y repetitivo**” sea el archivo (HTML, SVG, JSON, TXT por ejemplo) y se vuelve **marginal** en archivos con **contenido más denso o variado** como JavaScript.

8. Niveles de compresión

¿Qué gana en tamaño al pasar de nivel $1 \rightarrow 6 \rightarrow 9$ (gzip) y $5 \rightarrow 11$ (brotli)? ¿Justifica ese ahorro adicional el mayor costo de CPU/latencia? Identifique el punto de rendimientos decrecientes.

Subir de nivel de **gzip** de 1 a 6 sí trae una ganancia real en tamaño pero de 6 a 9 casi no cambia en nada. En el **app.js** por ejemplo **el nivel 1 deja el archivo en 102649 bytes y el nivel 6 lo baja a 84014 bytes**, pero en **nivel 9 apenas lo mejora a 83592 bytes**. Sin embargo el tiempo sí sigue subiendo bastante ya que pasa de **4.8 ms en nivel 1 a 13.7 en nivel 6 y hasta 22.9 ms en nivel 9**. Ahí ya se ve que ese último salto de nivel de 6 a 9 casi no aporta nada en espacio pero sigue costando CPU.

Con **Brotli** el patrón es todavía más extremo y ahí es donde se ve mejor el “**punto de rendimiento decrecientes**” que pide la pregunta. En el **datos.json** la **calidad 5 deja el archivo en 13961 bytes** con un **tiempo de 5.5 ms**, mientras que **la calidad 11 en vez de mejorar el tamaño lo empeora un poco (16435 bytes)** y **el tiempo se dispara a 608 ms**. Algo similar pasa en el **app.js** donde **la calidad 11 sí mejora el tamaño (69572 bytes)** pero **el tiempo se va hasta 418 ms**, es decir **casi 34 veces más lento que la calidad 5** para ganar **apenas un 5% adicional de compresión**.

Con estos datos se puede concluir que:

- Gzip nivel 6 es el punto de equilibrio ideal, porque el nivel 9 casi no gana espacio y sí gasta CPU
- Brotli calidad 11 solo se justifica en contenido que se comprime una sola vez y se sirve muchas veces (contenido estático precomprimido), nunca al vuelo
- Broly calidad 5 es el punto de equilibrio recomendado para compresión dinámica en tiempo real

9. Tipos de archivo ya comprimidos

¿Por qué los recursos ya comprimidos (JPEG, PNG, MP4, ZIP) no se benefician – o incluso crecen – al comprimirlos? Sustenta con datos.

Para el desarrollo del parcial estos recursos fueron excluidos de la compresión estos recursos precisamente porque **ya vienen comprimidos internamente** desde que se crearon (JPEG con su propio algoritmo de compresión de imagen, ZIP con DEFLATE ya aplicado sobre

su contenido), entonces cuando gzip o Brotli intentan buscar patrones repetidos para comprimir, ya no queda casi ninguna redundancia que aprovechar.

Si no se hubiera hecho esa exclusión, lo normal es que el **archivo resultante quedará un poco más pesado que el original**, porque el algoritmo de compresión agrega su propio **encabezado de control** aunque no logre eliminar ninguna redundancia real. Por eso comprimir estos tipos de archivo es contraproducente, ya que en el **mejor caso no cambia nada y en el peor caso el archivo pesa más** y además se **gastó CPU del servidor sin ninguna ganancia a cambio**.

10. Impacto en CPU y ancho de banda

Discuta el equilibrio entre ahorro de ancho de banda y consumo de CPU del servidor, y cómo cambia bajo concurrencia alta.

Los datos muestran un **trade-off** bastante claro entre ahorrar ancho de banda y gastar CPU. Por un lado están los ahorros de tamaño que son enormes, como el **lorem.txt** que pasa de **1190700 bytes a solo 132 bytes con Brotli calidad 11**, un ahorro de prácticamente el 100%. Pero ese mismo resultado tardó **59.7 ms** en generarse, comparado con los **2 ms** que tarda servir el archivo sin comprimir. Es decir que **gana muchísimo en ancho de banda pero a costa de un tiempo de procesamiento notablemente mayor**.

Este **trade-off** se vuelve crítico bajo concurrencia alta. Si un servidor recibe muchas peticiones simultáneas por segundo y cada una de ellas tarda decenas o cientos de milisegundos solo en comprimir (como se vio con **Brotli calidad 11 en datos.json, 608 ms**), el servidor puede terminar con todos sus **núcleos de CPU ocupados** comprimiendo en vez de atendiendo nuevas conexiones, generando una cola de espera que termina siendo peor que el ahorro de ancho de banda que se buscaba. En cambio con niveles bajos (**gzip nivel 1 o brotli calidad 5**) el costo por petición es de apenas unos pocos milisegundos, algo mucho más **sostenible** cuando hay muchas peticiones al mismo tiempo.

Por eso en producción real la recomendación es buscar un balance intermedio y no simplemente activar el nivel máximo de compresión pensando que “**más es mejor**”, porque ese nivel máximo puede convertirse en el **cuello de botella real del servidor** en momentos de alto tráfico.

11. Contenido estático vs dinámico

Argumente cuándo conviene precomprimir en disco (brotli 11 servido con mod_headers) frente a comprimir al vuelo, y proponga una configuración recomendada para producción justificando algoritmo y nivel por tipo de contenido

Con estos datos queda muy claro que **Brotli calidad 11 nunca debería usarse para comprimir al vuelo**, porque tiempos como **419 ms o 608 ms** por petición son completamente **inviabiles si esa compresión se repite en cada solicitud del usuario**. Sin embargo esa misma calidad 11 sigue siendo **perfectamente válida si se aplica una sola vez y no en cada petición**, esto es precisamente lo que se conoce como **precompresión en disco**.

La lógica de precomprimir es sencilla, ya que en vez de que Apache comprima el archivo cada vez que alguien lo pide, se genera de antemano una **versión .br del archivo** (por ejemplo [app.js.br](#)) y luego con el módulo **mod_headers** se le indica al servidor que **si esa versión ya existe la sirva directamente**, sin gastar CPU comprimiendo en tiempo real. De esa forma se logra el mejor ratio posible (el de calidad 11) sin pagar su costo de tiempo en cada petición, porque ese costo ya se pagó una sola vez al momento de generar el archivo.

Con base en todo lo anterior la configuración recomendada para producción sería:

Tipo de contenido	Recomendación
Archivos estáticos que casi no cambian (CSS, JS de librerías, HTML fijo)	Precomprimir con Brotli calidad 11 y servir con mod_headers

Contenido dinámico generado en cada petición (HTML armado por backend, JSON de una API)	Comprimir al vuelo con Brotli calidad 5 o gzip nivel 6 como respaldo para clientes que no soporten Brotli
Imágenes, video, ZIP y demás binarios ya comprimidos	Excluir totalmente de la compresión, tal como se configuró con SetEnvIfNoCase